

AS

add to db and analyse

AS

AS

AS

AS

AS

AS

AS

AS

AS

AS

AS

AS

- AS

AS

AS

- AS

AS

AS

AS

- AS

AS

AS

```
└─ resources
  ├── Hindsight GitHub
  ├── Hindsight Docs
  ├── Hindsight Cloud
  ├── Vectorize Agent Memory
  └── Hindsight Community
```

The content guide also provides six prompt-driven stages covering titles, article generation, LinkedIn content, video scripting, and thumbnail generation. [Hackathon Content Guide](#)

Key analysis

The **weak approach** would be building another generic chatbot and attaching Hindsight somewhere in the backend.

The documents are clearly pushing toward this story:

Without memory → generic agent
With memory → agent that accumulates context, learns preferences/domain knowledge, and changes its behavior.

That should be the core of whatever project you build. The judging criteria make Hindsight worth **25%**, while innovation is **30%**, so the project needs both a meaningful problem and a visibly important memory mechanism. [HackwithHyderabad 3.0 Problem S... +1](#)

I don't have a connected database/schema in this chat that I can directly write these records into, so I haven't falsely claimed that they were inserted. The extracted structure above is ready to map into your DB.

    ...  Sources

whsat shall we pick

Buddy, based on the **actual judging criteria and problem statements**, I would pick:

 Incident Response Agent

Not another generic chatbot. Build an **AI agent that remembers production incidents, root causes, fixes, and what worked previously — then uses that memory during the next incident.**

The problem statement explicitly lists Incident Response as a strong memory-based engineering use case: remembering past incidents, root causes, resolution steps, and runbooks, then using that history for similar incidents. [HackwithHyderabad 3.0 Problem S...](#)

The concept

"An AI SRE that remembers every incident."

Example:

Incident #1

Database latency spike
→ Agent investigates
→ Root cause: missing index
→ Fix: added composite index
→ Resolution: 14 minutes

Two weeks later:

Incident #2

Database latency spike

A normal LLM starts from zero.

Your agent says:

"This looks similar to the database latency incident on September 12. That incident was caused by a missing composite index. The team resolved it by adding the index. Current logs also show a similar query pattern."

That's the **Hindsight moment**

Why this fits the judging

Criterion	How our project attacks it
Innovation – 30%	Goes beyond chatbot → persistent incident intelligence
Hindsight – 25%	Memory is the actual product, not decoration
Technical – 20%	Logs + agent + retrieval + memory + reasoning
UX – 15%	Incident dashboard + investigation timeline
Impact – 10%	Clear DevOps/SRE business use case

The official guide specifically says memory should be central and judges should see the difference between a generic agent and one that improves through previous interactions. [HackwithHyderabad 3.0 Problem S...](#)

What we should build

Product name

RecallOps

"Your incident response agent that doesn't forget."

Architecture



Killer demo

Don't waste 5 minutes explaining architecture.

Show this:

Minute 0

New incident: `API latency > 3 seconds`

Agent investigates.

No relevant historical incidents found.
Beginning investigation...

Then create/resolve several incidents.

Store:

Incident
Root Cause
Symptoms
Commands Used
Fix
Outcome
Time to Resolution

Then trigger a **similar incident**.

Now:

⚠ Similar historical incident detected

Previous incident: INC-1042
Similarity: 87%

Root cause:
Redis connection exhaustion

Previous resolution:
Increase connection pool + restart affected service

Previous resolution time:
11 minutes

Recommended next steps:
1. Inspect Redis connection count
2. Compare pool configuration
3. Check service restart history

That is much stronger than:

"Here is an AI chatbot that can answer DevOps questions."

The second idea I'd consider

Code Review Memory Agent

This is also directly supported by the problem statement.

The agent remembers:

- your team's coding standards
- previous review comments
- architectural preferences
- recurring mistakes

Then future reviews become personalized rather than generic.

Example:

```
PR #31
PR #38
PR #44
PR #47
"
```

This is technically interesting and fits your CSE background extremely well.

What I would NOT pick

I'd avoid:

- ✗ Generic AI tutor
- ✗ AI study assistant
- ✗ AI quiz generator
- ✗ Generic personal assistant
- ✗ "ChatGPT but with memory"
- ✗ Random multi-agent platform

The problem statement explicitly tells teams to avoid student-centric projects such as AI tutors, group-project managers and quiz generators.

And the guide says a **tight, polished workflow beats a sprawling prototype**.

My pick for your team

RecallOps — Memory-Powered Incident Response Agent

Keep the scope brutally tight:

```
ONE PERSONA
SRE / DevOps Engineer

ONE PROBLEM
Recurring production incidents

ONE MEMORY
Past incidents + resolutions

ONE CORE DEMO
Same/similar incident happens again

ONE WOW MOMENT
Agent recalls exactly how the team solved it before
```

That's the story.

And importantly, the content guide wants exactly this kind of **before → after → memory changes behavior** story in the article and video.

If we choose this, the next step should be designing the complete MVP: **features → architecture → Hindsight memory schema → agents/tools → database → UI → demo scenario → 3-minute pitch**.

Sources

📄 🔍 📌 🔄 ⋮

locked then break down the root cause of and problem

Locked. Now let's **tear the problem apart properly** before we write a single line of code.

The official problem statement gives us the direction: an Incident Response Agent should remember **past incidents, root causes, resolution steps, and which runbooks worked**, then use that history when similar incidents happen again.

RecallOps — Root Problem Breakdown

1. The actual problem

Production incidents repeat, but the organization's knowledge doesn't.

When something breaks in production, engineers usually need to answer:

"Have we seen this before, and how did we fix it?"

The information may exist across:

- incident tickets
- logs
- monitoring alerts
- Slack/Teams conversations
- postmortems
- runbooks
- Git commits
- deployment history
- engineers' personal knowledge

The problem isn't simply **lack of information**.

The problem is **lack of usable historical memory at the moment of an incident**.

2. Root cause tree



That is the **systemic problem** we are attacking.

3. Root causes

There isn't one root cause. There are several layers.

Root Cause #1 — Organizational memory is fragmented

Previous incident knowledge doesn't naturally become usable context for the next incident.

For example:

```
INC-1042
API latency
↓
Root cause discovered
↓
Redis connection exhaustion
↓
Fix applied
↓
Incident closed
```

Two months later:

```
INC-1187
API latency
```

The engineer may not immediately connect the two.

RecallOps creates that connection through persistent memory.

Root Cause #2 — Incident knowledge is mostly reactive

Teams often document what happened **after** the incident.

The useful information becomes:

```
What happened?
Why?
What fixed it?
What didn't work?
```

But during the next incident, an engineer still has to manually find that historical knowledge.

So the issue isn't documentation alone.

It's **retrieval + contextualization at the right moment**

Root Cause #3 — Similar incidents don't look identical

This is important for our project.

A future incident might not have exactly the same wording.

Previous:

```
"API requests timing out"
```

Current:

"Gateway response latency increased"



They could potentially share a root cause.

A simple keyword search can miss that relationship.

That's where our agent + memory architecture becomes interesting.

Root Cause #4 — Engineers repeat investigation work

Suppose engineers previously tested:

1. Restart service	+ Didn't solve it
2. Increase timeout	+ Didn't solve it
3. Check Redis pool	+ Found issue
4. Increase pool size	+ Solved it



A future engineer shouldn't have to rediscover:

✗ Don't waste time on #1
✗ Don't waste time on #2
✓ Investigate #3



The historical **failed and successful approaches** are valuable memory.

4. The deeper problem

Here's the strongest way to frame it:

Incident response teams don't just lack data. They lack accumulated operational memory.

That's our entire product thesis.

A conventional AI agent:

```
Incident
↓
LLM
↓
Reasoning
↓
Answer
```



RecallOps:

```
Incident
↓
Current evidence
+
Historical incidents
+
Past resolutions
+
Failed approaches
+
Runbooks
↓
Hindsight
↓
Agent reasoning
↓
Context-aware response
```



5. What Hindsight actually does

This is where we need to be careful.

Hindsight cannot just be a database that we slap onto the project.

The official judging criteria give Hindsight memory 25%, and specifically ask whether memory is central to the value proposition and whether the agent improves over time. [HackwithHyderabad 3.0 Problem 5...](#)

So our memory lifecycle should look like:

```
INCIDENT
↓
Investigate
↓
Resolution
↓
Extract useful knowledge
↓
RETAIN
↓
HINDSIGHT MEMORY
↓
Future incident
↓
RECALL
↓
Compare with historical incidents
↓
Recommend investigation path
↓
Resolve
↓
RETAIN new knowledge
```



That's the loop.

6. The killer before/after

This is what I want our entire demo built around.

Without RecallOps

```
Engineer:
"API latency is 5 seconds."

Agent:
"Check CPU utilization,
network latency,
database performance,
cache..."
```

Generic troubleshooting.

With RecallOps

```
Engineer:
"API latency is 5 seconds."

RecallOps:

⚠ Similar historical incident detected

Previous Incident: INC-1842

Symptoms:
API latency > 3 seconds

Historical root cause:
Redis connection exhaustion

Previous investigation:
• CPU → normal
• DB → normal
• Network → normal
• Redis connections → abnormal

Previous resolution:
Increased connection pool size

Previous outcome:
Resolved in 11 minutes

Recommended investigation:
Check Redis connection utilization first.
```

That is our wow moment.

7. What makes this commercially meaningful

The problem statement explicitly tells teams to target workflows where a business could realistically pay for the solution, rather than building novelty projects. [HackwithHyderabad 3.0 Problem S...](#)

Our value proposition becomes:

```
Less repeated investigation
↓
Faster incident diagnosis
↓
Less engineering time wasted
↓
Institutional knowledge retained
↓
New engineers benefit from old incidents
```

We should **not claim actual percentage improvements** until we have measured them.

8. Our problem statement

Short version

Engineering teams repeatedly encounter similar production incidents, but the knowledge from previous incidents is fragmented across tickets, logs, runbooks, and postmortems. Engineers therefore spend valuable time rediscovering causes and solutions instead of reusing what the organization has already learned.

Product version

RecallOps is a memory-powered incident response agent that learns from resolved incidents and uses Hindsight to recall relevant historical failures, root causes, investigation paths, and successful resolutions when similar incidents occur.

9. The real enemy

Not:

- ✗ Production downtime alone
- ✗ Bad logs
- ✗ Lack of monitoring

✖ Lack of an LLM

The enemy is:

Repeated rediscovery.

An organization can have 1,000 incident reports and still behave like it has never seen the problem before.

RecallOps turns those past incidents into operational memory.

And that's the story I'd build the entire project around.

Sources

📄 🔍 ⬆️ ↺ ⋮

got it

Good. Then we're aligned.

RecallOps = persistent operational memory for incident response.

Next, we should lock the MVP scope and exact demo flow before touching architecture or code. That's where most hackathon projects become bloated trash.

📄 🔍 ⬆️ ↺ ⋮

can we keep any other name something interesting and inovative

Absolutely. "RecallOps" is functional but boring. For this project, the name should communicate memory + incidents + learning without sounding like another enterprise monitoring dashboard.

Here are stronger options:

Name	Idea behind it	Vibe
Remembra	Remember + infrastructure	★★★★★
OpsMind	Operational memory/brain	★★★★★
TraceMind	Learns from incident traces	★★★★★
ReSolve	Remembers how problems were solved	★★★★★
EchoOps	Past incidents echo into future response	★★★★★
IncidentIQ	Incident intelligence	★★★★★
RootRecall	Recalls previous root causes	★★★★★
OpsEcho	Operational knowledge echoes forward	★★★★★
SentryMind	Monitoring + memory	★★★★★
MemoraOps	Memory + operations	★★★★★
CausalOps	Focus on causes, not symptoms	★★★★★
PostMortem AI	Learns directly from incident history	★★★★★
OpsMemory	Extremely clear	★★★★★
ChronoOps	Uses historical operational context	★★★★★
ReMindOps	Re-mind + operations	★★★★★

My strongest 5

1. EchoOps

“Every incident leaves an echo.”

This is genuinely good for the demo story: an old incident **echoes into the future**.

2. RootRecall

“Remember the root cause. Resolve faster.”

Very directly communicates what the system does.

3. TraceMind

“Turn incident history into operational intelligence.”

More technical/startup-like.

4. CausalOps

“From symptoms to causes, from incidents to learning.”

Strong if we make root-cause analysis a major part of the product.

5. Remembra

"Infrastructure that remembers."

More brandable, less descriptive.

🔥 My pick: EchoOps

Because we can build the entire identity around the concept:



That gives us a **much better product narrative** than simply calling it an "Incident Response Agent."

I'd lock `EchoOps` unless you want me to generate 30–50 much more unconventional names—including names that sound like actual YC/AI infrastructure startups.

📄 🔍 ⬆️ ↺️ ⋮

take inspiration from the great ai model and give

Yes. If we're taking inspiration from **great AI/infrastructure model naming**, I wouldn't use another obvious `AI + Ops` name. The strongest names tend to be **short, memorable, metaphorical, and expandable**.

For our incident-memory agent, I'd explore names around **memory, traces, history, echoes, causes, learning, and resilience**.

Stronger naming direction

- **Memora** — memory-centric, clean AI-product feel
- **Nerva** — evokes a neural/central nervous system
- **Aegis** — protection/defense; strong infrastructure feel
- **Kairo** — time/history-oriented, modern startup sound
- **Syntra** — synthetic + intelligence/network feel
- **Cortex** — operational "brain" concept
- **Echora** — echo + memory
- **Retra** — retain + trace
- **Causely** — focused on finding causes
- **Traceon** — traces + operational intelligence
- **Revia** — revisit + intelligence
- **Mnema** — inspired by *mneme*, memory
- **Anamnesis** — literally associated with recollection/remembering; intellectually distinctive
- **Chrona** — historical/time dimension
- **Nexora** — next + memory/operations feel
- **Remnix** — remembrance + systems
- **Oryn** — short, abstract infrastructure brand
- **Veyra** — abstract, premium AI-product feel
- **Kora** — short and brandable
- **Eidon** — identity/knowledge feel

My top 7 for this project

1. Memora

Infrastructure that remembers.

2. EchoOps

Every incident leaves an echo.

3. RootRecall

Remember the cause. Resolve the incident.

4. CortexOps

The operational brain that learns from incidents.

5. TraceMind

Turn incident history into operational intelligence.

6. Mnema

Operational memory for systems that learn.

7. Causely

Find the cause. Remember the fix.

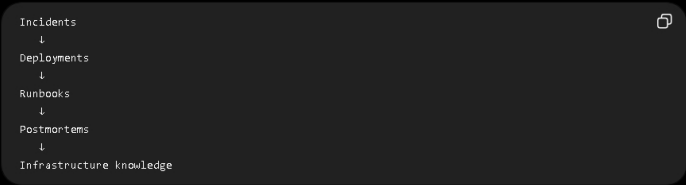
But here's the important part

I would **not** choose a name just because it sounds cool.

Our product has a very specific differentiator:

Incident → investigation → resolution → memory → future incident → recall → better investigation.

So the name should survive when we expand from incident response into:



That makes **Memora**, **EchoOps**, **TraceMind**, and **CortexOps** more future-proof than something overly narrow like **RootRecall**.

If you want the **OpenAI / Anthropic / DeepMind / Perplexity**-style naming philosophy specifically, I'd go one level more abstract and create **20 names that sound like a serious AI infrastructure company**, not a college hackathon project.

    ...

Memora locked

Locked. 🗝️

MEMORA

Tagline

"Infrastructure that remembers."

Product definition

Memora is a memory-powered incident response agent that learns from past production incidents and uses that operational memory to investigate and resolve future incidents more intelligently.

The core loop stays:

Incident → Investigate → Resolve → Remember → Recall → Improve

And Hindsight remains the central memory layer, which directly matches the competition's requirement that memory be central to the agent's value.  HackwithHyderabad 3.0 Problem Solving

Next, we should define **Memora's exact MVP features and user journey** before designing the architecture.

Sources

    ...

let's begin

MEMORA — Phase 1: Lock the MVP

We're not building a giant AIops platform. The official brief explicitly favors **one workflow, one persona, and one clear value proposition**.  HackwithHyderabad 3.0 Problem Solving

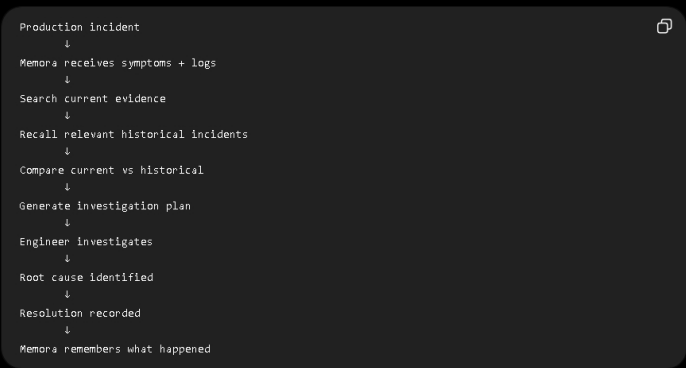
1. The user

Primary persona: DevOps / SRE engineer handling a production incident.

Their question is:

"Have we seen something like this before, and what worked?"

2. The one workflow



This directly implements the incident-response use case described in the problem statement: remembering incidents, root causes, resolution steps, and effective runbooks. [HackwithHyderabad 3.0 Problem Sol...](#)

3. MVP features

A. Incident intake

Engineer creates:

```
Incident ID
Service
Severity
Symptoms
Timestamp
Logs
Metrics
Recent deployment
```

Example:

```
INC-1047

Service: Payment API
Severity: SEV-1

Symptoms:
- API latency > 5s
- 32% requests timing out
- Error rate increasing

Recent deployment:
payment-api v2.4.1
```

B. Investigation Agent

Memora analyzes the current incident.

It should produce:

```
Current symptoms
Possible causes
Evidence
Recommended checks
Historical matches
Confidence / reasoning
```

But it must not pretend certainty.

C. Hindsight Memory

This is the heart of Memora.

We retain:

```
Incident
├─ symptoms
├─ root cause
├─ investigation steps
├─ failed approaches
├─ successful approaches
├─ resolution
├─ runbook used
├─ outcome
└─ lessons learned
```

The next incident can recall this information.

D. Historical similarity

Example:

```
CURRENT INCIDENT

Payment API latency
┆
┆
┆
MEMORA
┆
┆
Historical match: INC-0982
┆
┆
Similar symptoms:
87%
┆
┆
Previous root cause:
Database connection pool exhaustion
┆
┆
Previous successful fix:
Increase pool + rollback deployment
```

The 87% should only appear if we actually implement a defensible similarity calculation. No fake numbers for the demo.

E. Resolution capture

After the incident:

```
Root Cause:
Connection pool exhaustion

What worked:
Increased pool from 50 → 100

What failed:
Restarting API
Increasing request timeout

Runbook:
DB-CONNECTION-POOL-03

Resolution:
12 minutes
```

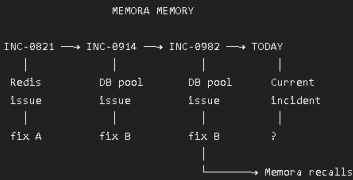
Then Memora sends the useful information into Hindsight.

That's how the system **gets better with experience** rather than simply retrieving static documents.

4. The killer feature

Memory Timeline

This should be one of our major UI elements.



Now the judge can **see the organization learning over time**.

That matters because the brief specifically says compelling demos should show the progression from generic behavior to personalized behavior through repeated interactions. [HackwithHyderabad 3.0 Problem 5...](#)

5. The MVP dashboard

Keep it to **5 screens/areas**:

① Overview

```
Active Incidents
Critical Incidents
Resolved Today
Historical Incidents
Memory Entries
```

② Incident Workspace

```
Incident
Logs
Metrics
Timeline
Agent Investigation
```

③ Memory

```
Historical incidents
Root causes
Successful fixes
Failed approaches
Runbooks
```

④ Investigation

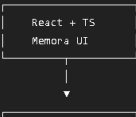
```
Current evidence
Historical matches
Recommended checks
Agent reasoning
```

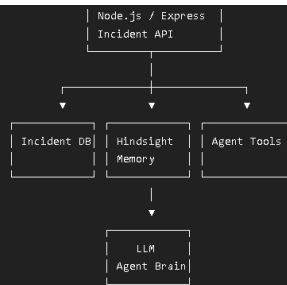
⑤ Post-Incident

```
Root cause
Resolution
What worked
What failed
Lessons learned
[Save to Memora]
```

6. The architecture

Our first architecture should be:





Your existing strength with **React + TypeScript + Node/Express + Supabase** makes this a practical stack rather than introducing unnecessary technology.

7. What we deliberately DON'T build

For MVP:

- ✗ Kubernetes management
- ✗ Automatic production remediation
- ✗ 20 different agents
- ✗ Full observability platform
- ✗ Real PagerDuty replacement
- ✗ Real cloud infrastructure control
- ✗ Massive multi-tenant enterprise system
- ✗ Fancy predictive claims

Those are distractions.

Our first version needs to nail:

“Something broke. Memora remembers how we solved something similar before.”

8. The demo story

This is the part I care about most.

Scene 1 — Memora has no memory

Incident:
Payment API latency

Memora:
“I don't have relevant historical context.
I'll begin a fresh investigation.”

Scene 2 — Resolve it

We deliberately create a realistic investigation:

CPU → normal
Network → normal
DB connections → abnormal

Root cause:
DB connection pool exhaustion

Resolution:
Increase pool size

Memora remembers it.

Scene 3 — New incident

Later:

Payment API latency
+ request timeouts
+ DB connection spike

Scene 4 — Memory activates

MEMORA FOUND RELEVANT HISTORY

INC-1047

Similar symptoms:
Database connection exhaustion

Previous investigation:
✓ CPU checked
✓ Network checked
✓ DB pool identified

Previous successful resolution:
Increase connection pool

Previous failed approach:
API restart

Scene 5 — Agent adapts

Instead of starting from zero:

"This incident resembles INC-1047. I recommend checking database connection utilization first because that was the root cause in the previous incident."

That's our product.

9. The fundamental data model

Before coding UI, we'll define these entities:

```
users
incidents
incident_events
incident_logs
investigations
investigation_steps
root_causes
resolutions
runbooks
memory_records
agent_actions
```

But we shouldn't blindly create all of them yet.

Next we should design the exact database schema + Hindsight memory schema, including what gets stored, what gets recalled, and what the agent is allowed to do. That's the foundation everything else depends on.

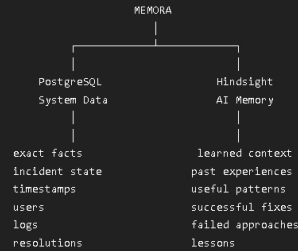
    ...  Sources

proceed

Memora — Database & Memory Architecture

1. Core principle

We separate **system-of-record data** from **AI memory**.



PostgreSQL answers: *What happened?*

Hindsight answers: *What should the agent remember from what happened?*

That separation prevents us from turning Hindsight into a messy replacement for the application database.

2. Database schema

incidents

The central entity.

```
SQL
id
incident_key
title
service
severity
status
description
detected_at
resolved_at
created_by
created_at
updated_at
```

Example:

```
INC-1047
Payment API Latency
SEV-1
Resolved
```

incident_events

Everything that happened during the incident.

```
SQL
id
incident_id
event_type
message
source
```

```
metadata
created_at
```

Possible `event_type`:

```
ALERT
INVESTIGATION
OBSERVATION
ACTION
ERROR
RESOLUTION
```

Example:

```
OBSERVATION
"DB connection count reached 98%"

ACTION
"Checked connection pool configuration"

ERROR
"Restarting API did not reduce latency"
```

This becomes extremely useful when reconstructing the investigation timeline.

3. `incident_evidence`

Current evidence shouldn't be mixed with historical memory.

```
SQL
id
incident_id
type
name
value
source
timestamp
metadata
```

Examples:

```
metric
api_latency
4.82s

metric
db_connections
98%

log
payment-api
"connection pool exhausted"

deployment
payment-api
v2.4.1
```

4. `investigations`

Tracks the agent's investigation.

```
SQL
id
incident_id
agent_id
status
summary
started_at
completed_at
```

5. `investigation_steps`

This is important because **failed approaches are knowledge too**.

```
SQL
id
investigation_id
step_number
hypothesis
action
observation
result
status
created_at
```

Example:

```
1
Hypothesis:
CPU saturation

Action:
Check CPU metrics

Observation:
CPU = 41%
```

```
Result:
CPU unlikely to be root cause

status:
RULED_OUT
```

Then:

```
3
Hypothesis:
DB connection exhaustion

Action:
Inspect connection pool

Observation:
98% utilization

Result:
Strong evidence

status:
CONFIRMED
```

6. resolutions

```
SQL

id
incident_id
root_cause
resolution_summary
runbook_id
resolution_time_minutes
verified
created_at
```

Example:

```
Root cause:
Database connection pool exhaustion

Resolution:
Increase pool size from 50 → 100

Runbook:
DB-CONNECTION-POOL-03

Resolution time:
12 minutes
```

7. runbooks

```
SQL

id
name
service
description
steps
version
created_at
updated_at
```

The agent can use runbooks as **procedural knowledge**, while Hindsight stores experience from actually using them.

8. Hindsight memory model

This is where Memora becomes different from a normal incident dashboard.

We shouldn't blindly retain every log.

We retain **experience**.

Memory object

```
Incident experience

Service:
Payment API

Symptoms:
- high latency
- request timeouts
- DB connection spike

Root cause:
Connection pool exhaustion

Investigation:
- CPU ruled out
- network ruled out
- DB pool confirmed

Failed approaches:
- API restart
- timeout increase

Successful resolution:
```

- Increased connection pool

Runbook:

DB-CONNECTION-POOL-83

Outcome:

Resolved

Lesson:

When payment API latency coincides with high DB connection utilization, inspect the connection pool before restarting the service.

This is the kind of information the next investigation can actually use.

9. Retain vs Recall

This distinction is critical.

RETAIN

After an incident:

```
Incident resolved
↓
Extract experience
↓
Normalize useful facts
↓
Hindsight RETAIN
```

We retain:

- root cause
- symptoms
- investigation path
- failed approaches
- successful approaches
- resolution
- runbook
- lessons learned

RECALL

When a new incident arrives:

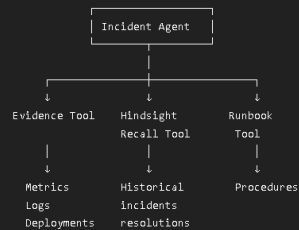
```
New Incident
↓
Extract symptoms
↓
Hindsight RECALL
↓
Relevant historical experiences
↓
Agent reasoning
↓
Investigation plan
```

The agent shouldn't simply dump old incidents onto the engineer.

It should **interpret the recalled experience against the current evidence**.

10. Agent architecture

Start with **one primary agent**, not seven fake agents.



Later we can split responsibilities if the code actually needs it.

11. Agent decision loop

```
NEW INCIDENT
↓
Understand symptoms
↓
Collect evidence
↓
Recall historical memory
↓
Compare past vs current
↓
Generate hypotheses
↓
Rank investigation steps
↓
Execute / request checks
↓
```

```
Observe results
↓
Update hypothesis
↓
Identify root cause
↓
Recommend resolution
↓
Capture outcome
↓
RETAIN EXPERIENCE
```

This is the actual intelligence loop.

12. The most important UI

The UI should make memory visible.

Historical Memory panel

```
MEMORA MEMORY

Similar incident found

INC-1047
Payment API

Root Cause
DB Connection Pool Exhaustion

Previous successful fix
Increase connection pool

Previous failed approaches
• Restart API
• Increase timeout

[View Incident]
```

That panel is our **Hindsight showcase**.

13. API contract

We'll keep the backend simple.

```
POST /api/incidents
GET /api/incidents
GET /api/incidents/:id

POST /api/incidents/:id/investigate

GET /api/incidents/:id/timeline
GET /api/incidents/:id/memory

POST /api/incidents/:id/evidence
POST /api/incidents/:id/resolve

POST /api/memory/retain
POST /api/memory/recall
```

The important endpoints are:

```
/investigate
/memory/recall
/memory/retain
```

Those three form the heart of Memora.

14. Our first demo dataset

We shouldn't wait for real production integrations.

Create a realistic synthetic incident history:

```
INC-1001
Redis connection exhaustion

INC-1002
Payment API timeout

INC-1003
Database connection pool exhaustion

INC-1004
Checkout service latency

INC-1005
Redis memory pressure

INC-1006
Payment API latency
```

Then introduce:

```
INC-1007
Payment API latency + DB connection spike
```

Memora should discover that **INC-1007 resembles previous incidents**, retrieve the relevant experiences, and

guide the investigation.

The official guide explicitly recommends realistic data because it makes the project feel like a real system rather than a toy demo. [HackwithHyderabad 3.0 Problem S...](#)

15. What we build first

Phase 1

Supabase/PostgreSQL

+

Backend API

+

Hindsight

+

LLM

Phase 2

React dashboard

+

Incident workspace

+

Memory visualization

Phase 3

Agent tools

+

Investigation workflow

+

Retain/recall loop

Phase 4

Demo dataset

+

Before/after demonstration

+

Failure cases

+

Polish

Phase 5

Article

LinkedIn post

3-minute video

Architecture diagram

The content guide specifically calls for showing **real retain/recall behavior**, project screenshots, and an architecture diagram. [Hackathon Content Guide](#)

The one sentence we build everything around

Memora gives incident response teams a persistent operational memory, allowing an agent to learn from previous failures and use that experience when the next incident happens.

Next: we should design the actual Hindsight `retain()` and `recall()` strategy and the agent prompts. That is the part that determines whether Memora genuinely uses memory or just has a memory-themed UI.

Sources



YourSelf Media

Ad

Build Apps with Emergent

Emergent is world's first agentic full stack AI platform to build. Start for free.

Today 8:05 PM

next

Memora — Phase 2: Hindsight Retain/Recall Design

The official brief says the strongest projects show the agent **improving over multiple interactions**, recalling context from earlier incidents, and adapting from what happened previously. [HackwithHyderabad 3.0 Problem S...](#)

So our memory architecture will have **two explicit operations**:

RETAIN → learn from a resolved incident

RECALL → use that learning during a new incident

1. What Memora should remember

Do **not** retain raw logs as the primary memory.

Retain the **experience extracted from the incident**:

```
Incident Experience
├── Service
├── Symptoms
├── Error patterns
├── Root cause
├── Investigation path
│   ├── what was checked
│   ├── what was ruled out
│   └── what was confirmed
├── Failed approaches
├── Successful approaches
├── Resolution
├── Runbook
├── Outcome
└── Lesson learned
```

Example memory

```
Service:
payment-api

Symptoms:
high latency + request timeouts

Root cause:
database connection pool exhaustion

Evidence:
connection utilization reached 98%

Ruled out:
CPU saturation
network latency

Failed approaches:
API restart
timeout increase

Successful resolution:
increase DB connection pool

Runbook:
DB-CONNECTION-POOL-03

Lesson:
When payment-api latency coincides with high DB
connection utilization, inspect the connection pool first.
```

That is valuable memory.

2. RETAIN pipeline

When an incident is resolved:

```
Resolved Incident
┆
┆ Collect investigation
┆
┆ Extract important facts
┆
┆ Remove noise
┆
┆ Create experience
┆
┆ Hindsight RETAIN
```

The agent should **not automatically remember everything**.

We need a memory extraction step.

Retain prompt concept

```
You are Memora's incident-memory extractor.

Analyze this resolved incident.

Extract only information that could help
diagnose or resolve a future incident.

Capture:

1. Service
2. Symptoms
3. Important evidence
4. Root cause
5. Investigation steps
6. Failed approaches
7. Successful approaches
8. Resolution
9. Runbook used
10. Generalizable lesson

Do not invent information.

Do not store temporary conversational details.

Return structured incident experience.
```

The important instruction is:

Do not invent information.

If the incident never established a root cause, memory must say that rather than hallucinating one.

3. Recall pipeline

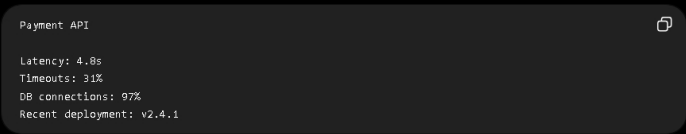
When a new incident starts:



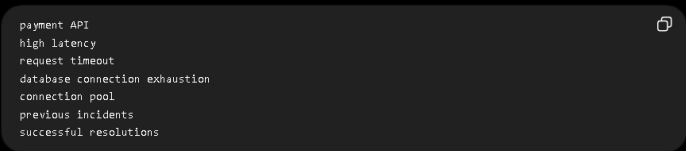
The agent should not blindly trust recalled incidents.

4. Recall query

Suppose the current incident is:



The memory query becomes something like:

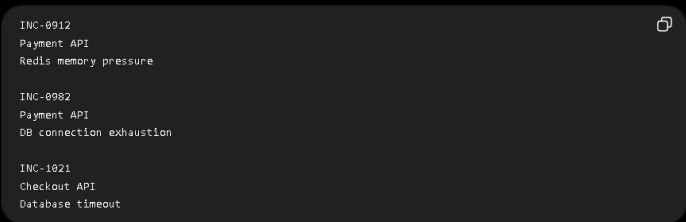


Then Hindsight returns potentially relevant experiences.

5. Relevance evaluation

This is important.

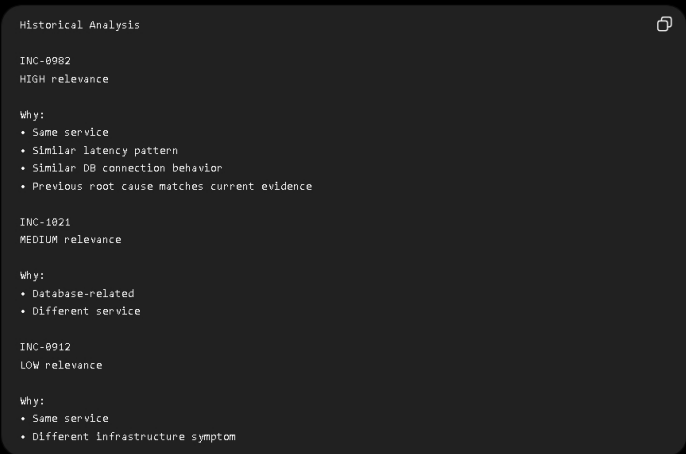
Suppose memory returns:



Memora shouldn't say:

"Here are three incidents."

Instead:



That gives us an explainable memory system.

6. Memory should change the investigation

This is the **most important part**.

Without memory

Agent:

1. Check CPU
2. Check memory
3. Check network
4. Check database
5. Check Redis
6. Check deployment

Generic troubleshooting.

With memory

Agent:

Historical incident INC-0982 had similar symptoms.

Previous investigation ruled out:

- ✓ CPU
- ✓ Network

The root cause was DB connection pool exhaustion.

Current evidence also shows:

DB connection utilization = 97%

Recommended next step:

→ Inspect DB connection pool.

Now memory actually changes behavior.

7. The agent's investigation prompt

Our core reasoning prompt should conceptually be:

You are Memora, an incident response agent.

Your job is to help investigate production incidents.

You have access to:

1. Current incident evidence
2. Historical incident memories
3. Investigation tools
4. Runbooks

Use historical memories as evidence, not as unquestionable truth.

For each relevant historical incident:

- explain why it is relevant
- identify similarities
- identify differences
- identify previously successful approaches
- identify previously failed approaches

Then create a prioritized investigation plan.

Prioritize checks that are supported by:

- current evidence
- relevant historical incidents
- available runbooks

Never claim a root cause unless the evidence supports it.

Clearly distinguish:

- observed facts
- historical evidence
- hypotheses
- confirmed root cause.

That distinction will make the system much more credible.

8. Memory confidence

We should visually distinguish:

OBSERVED

HISTORICAL

HYPOTHESIS

CONFIRMED

Example:

OBSERVED
DB connections: 97%

HISTORICAL
INC-0982 had similar symptoms.

HYPOTHESIS

Connection pool exhaustion may be responsible.

INVESTIGATION

Inspect pool utilization.

CONFIRMED

Pool exhausted at 100 connections.

ROOT CAUSE

Database connection pool exhaustion.

This prevents the agent from turning a remembered possibility into a fake fact.